Parallel Alpha-Beta Search for Chess: Root Splitting and a Hybrid Work Queue

Carter Griffin, Luke [Lname], Bryce [Lname]
School of Informatics, Computing & Cyber Systems
Northern Arizona University, Flagstaff, AZ, U.S.A.
{*[id1], [id2], [id3]*}*@nau.edu*

Algorithm 1 Sequential negamax with alpha-beta pruning

```
1: function SEARCH(p, d, α, β)
2:     if d = 0 then return EVAL(p)
3:     for m ∈ MOVES(p) do
4:         p' ← MAKE(p, m)
5:         s ← −SEARCH(p', d − 1, −β, −α)
6:         if s ≥ β then return β
7:         α ← max(α, s)
8:     return α
```

***Abstract*—Alpha-beta pruning makes minimax game-tree search practical, but its pruning depends on bounds produced by earlier siblings, which forces a partial ordering on the search. This project implements a sequential negamax alpha-beta search for chess in C and three pthreads-based parallel variants: static root splitting, dynamic root splitting through a shared FIFO work queue, and a hybrid that extends the work queue below the root in the manner of the Young Brothers Wait Concept. Each variant is measured against the sequential baseline for wall time, nodes searched, and speedup on up to 28 hardware threads.**

I. Project Idea

A chess engine selects a move by searching the tree of positions reachable from the current one and scoring the leaves with an evaluation function. Minimax search visits every node to a fixed depth. Alpha-beta pruning skips subtrees that cannot change the result, and with good move ordering it searches roughly the square root of the minimax node count [**?**]. Negamax is the same algorithm written with a single maximizing case, using $\max(a, b) = -\min(-a, -b)$ (Algorithm 1).

The difficulty for parallelism is in lines 5–7. The window (α, β) passed to each child depends on the scores of the children searched before it. Searching siblings concurrently means searching them with wider windows than the sequential algorithm would use, and so searching nodes the sequential algorithm would have pruned.

The project builds four versions of the search on a shared core (board representation, move generation, material evaluation):

1) **Sequential.** Algorithm 1, one thread. All speedups are reported relative to this version.
2) **Static root split.** The legal moves at the root are dealt round-robin to T threads before the search begins. Each thread searches its moves with a private α. Results are combined after a join.
3) **Dynamic root split.** Root moves are placed in a shared FIFO queue. Idle threads take the next move. The best score found so far is shared, so moves taken later are searched with a tighter window (Algorithm 2).
4) **Hybrid.** The work queue of version 3 extended below the root. At a node, the first move is searched by the owning thread alone; once it returns a bound, the remaining moves are pushed to the shared queue as tasks. This follows the Young Brothers Wait Concept [**?**] and reuses the queue from version 3 instead of a per-node split-point structure (Algorithm 3).

Algorithm 2 Dynamic root split (worker thread)

```
1: shared: queue Q of root moves, score α*, lock L
2: while POP(Q, m) do
3:     a ← α*
4:     p' ← MAKE(root, m)
5:     s ← −SEARCH(p', d − 1, −∞, −a)
6:     LOCK(L); α* ← max(α*, s); UNLOCK(L)
```

Algorithm 3 Hybrid split at node n (sketch)

```
1: s_0 ← −SEARCH(child 0 of n)      ▷ eldest brother, sequential
2: update n.α; if cutoff then return
3: if n.depth ≥ D_split then
4:     push children 1..k of n to Q as tasks
5:     help process tasks from Q until all children of n finish
6: else
7:     search children 1..k sequentially
8: on any child result ≥ n.β: set n.abort
```

II. Anticipated Challenges

The challenges follow from the dependence of each sibling's window on earlier siblings, and each version is affected differently.

Search overhead. Any node searched in parallel is searched with less information than in the sequential order. Static splitting has the most of this, since no thread sees another's α. The total node count at T threads, divided by the sequential node count, measures this cost directly.

Load imbalance and idle time. Subtree sizes under different root moves vary by orders of magnitude. The first move in a well-ordered list is usually the largest. Under a static split, one thread can hold most of the work while the others finish early and wait at the join.

Limited parallelism at the root. A typical middlegame position has 30 to 40 legal moves. Root splitting cannot use more threads than root moves, and in practice it is limited by the largest single subtree. This is the motivation for the hybrid.

Synchronization and contention. The shared queue and shared α^* are touched by every thread. In the hybrid, task pushes and pops occur at every split node, so the lock on Q is a likely bottleneck. Split depth D_{split} trades task granularity against contention.

Cutoff propagation. In the hybrid, a beta cutoff at one child makes the remaining sibling tasks useless. Those tasks must be cancelled or abandoned through a flag that running threads check, without adding a lock to every node.

Correctness and nondeterminism. Parallel versions may visit nodes in different orders across runs. Every version must return the same root score as the sequential search at the same depth, which serves as the correctness test.

III. Proposed Solution

The code is written in C with POSIX threads and built with gcc on Ubuntu. Board representation and legal move generation form a fixed core shared by all four versions, and the evaluation is material count only, so that measured differences come from the search and not from the evaluation. Move ordering (captures first) is the same across versions.

Static splitting uses one `pthread_create` per thread and a join. Dynamic splitting uses a mutex-protected array queue and a mutex-protected α^*; a variant reading α^* through a C11 atomic is a possible refinement. The hybrid adds a task structure (position, depth, window, parent pointer, abort flag) to the same queue, an atomic counter of outstanding children per split node, and a minimum split depth D_{split} chosen by experiment.

Table I
Sequential baseline, 1 thread

Position	Depth	Nodes	Time (s)
Start position	–	–	–
Middlegame 1	–	–	–
Middlegame 2	–	–	–
Endgame	–	–	–

Experiments use a fixed set of test positions in FEN notation, searched to a fixed depth. For each version and $T \in \{1, 2, 4, 8, 16, 28\}$ the measurements are wall time, total nodes, speedup $S_T = t_{seq}/t_T$, efficiency S_T/T, and search overhead N_T/N_{seq}. Each configuration is run several times and the median is reported.

IV. Preliminary Benchmark

Table I gives sequential search time on one thread for the test positions. [Numbers pending: sequential baseline not yet run.]

[Projection pending: target speedup per version at 16 and 28 threads, stated from the measured sequential times.]

V. Timeline

- Weeks 1–2: board, move generation, sequential search, test positions.
- Week 3: static and dynamic root splitting.
- Weeks 4–5: hybrid work queue, cutoff propagation, tuning D_{split}.
- Week 6: full measurements and final report.